

Technical Requirements Document (TRD)

Makeni Prayer Book: Catholic Companion

Chapter 1 — System Overview & Architecture

Version: 1.0

Document Type: Technical Requirements Document (TRD)

1. Purpose

This document defines the technical architecture, implementation strategy, and engineering standards for the Makeni Prayer Book: Catholic Companion platform.

The objective is to provide a scalable, offline-first, cross-platform mobile application that delivers the official Makeni Prayer Book through a premium digital reading experience while maintaining high performance, maintainability, accessibility, and privacy.

This document complements the PRD and SRS by describing **how the system will be implemented**, not **what features it provides**.

2. Architecture Principles

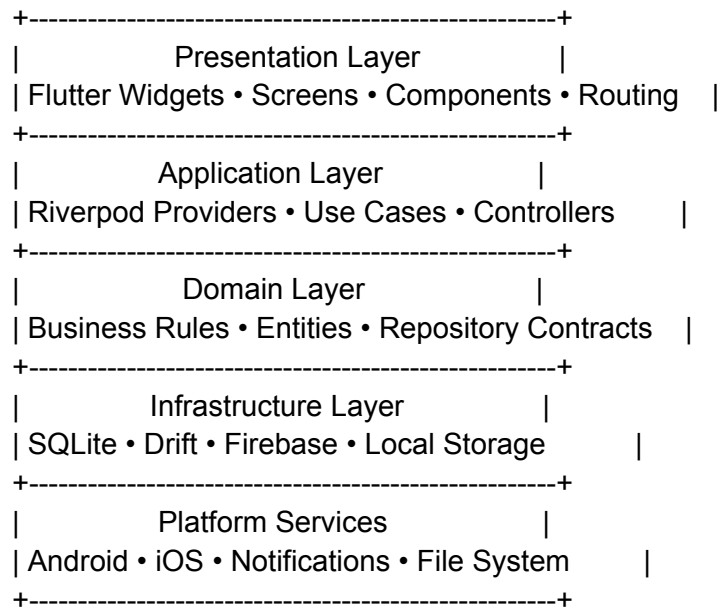
The platform shall be built according to the following engineering principles:

- Offline First
- Clean Architecture
- Feature-First Modularization
- Engine-Based Platform Architecture
- Privacy by Design
- Accessibility by Default
- Content-Driven Platform
- Performance-Oriented Development

- Testability
 - Scalability
-

3. High-Level Architecture

The application follows a layered architecture.



Each layer communicates only with adjacent layers, reducing coupling and improving maintainability.

4. Architectural Style

The project combines:

- Clean Architecture
- Feature-First Organization
- Repository Pattern
- Service Layer
- Engine-Based Modular Design

This hybrid approach allows independent development of features while keeping shared logic centralized.

5. Core Platform Engines

The application is built around reusable engines rather than tightly coupled feature logic.

Core Engines

- Content Engine
- Reading Engine
- Search Engine
- Intelligent Navigation Engine (INE)
- Recommendation Engine
- Rule Engine
- Reminder Engine
- Configuration Engine
- Theme Engine
- Typography Engine
- Accessibility Engine
- Consent Engine
- Analytics Engine
- Content Update Engine (Future)
- Synchronization Engine (Future)

Each engine exposes clear interfaces and is consumed by multiple feature modules.

6. Feature Modules

Primary feature modules include:

- Home Dashboard
- Library
- Reader
- Search
- Prayer Companion
- Personal Workspace
- Settings

- Reminder Center

Each module is independently organized and communicates through shared engines and repository interfaces.

7. Offline-First Strategy

Offline functionality is a foundational requirement.

The application shall:

- Store all core content locally.
- Store all user data locally.
- Execute searches locally.
- Generate recommendations locally.
- Schedule reminders locally.
- Persist settings locally.

Internet connectivity is required only for optional services such as analytics, feedback submission, and future content updates.

8. Content Architecture

Official content is not embedded directly into application code.

Instead, it is delivered through structured content packages containing:

- Metadata
- Prayers
- Hymns
- Categories
- Keywords
- Relationships
- Search indexes

This architecture supports future expansion without modifying the application itself.

9. Data Separation

The system maintains strict separation between:

Official Content

- Prayers
- Hymns
- Metadata
- Categories

User Content

- Favorites
- Bookmarks
- Notes
- Journal Entries
- Reading History
- Settings
- Collections

Official content remains immutable while user data remains fully editable.

10. Navigation Model

Navigation is managed through the Intelligent Navigation Engine (INE).

The INE provides:

- Standard routing
- Quick Jump
- Search navigation
- Continue Reading
- Navigation history
- Related content navigation

Future versions will support deep links and QR-based navigation without changing the routing architecture.

11. Security Philosophy

The application follows a privacy-first approach.

Core principles include:

- No authentication required for Version 1.
 - No cloud dependency for core features.
 - User data remains local by default.
 - Analytics are optional and opt-in.
 - Official content is protected from modification.
-

12. Scalability

The architecture is designed to evolve beyond the Makeni Prayer Book.

Future content packages may include:

- Catholic Bible
- Catechism
- Daily Mass Readings
- Rosary
- Novenas
- Saints
- Diocese Resources
- Multi-language Editions

No architectural redesign should be required to support these additions.

13. Development Goals

The engineering team shall prioritize:

- Readability
- Maintainability
- Testability
- Performance
- Accessibility

- Offline reliability
- Consistent user experience

Every implementation decision should align with the platform's guiding principle of **Sacred Simplicity**.

Chapter Summary

This chapter establishes the architectural foundation of the platform.

Subsequent TRD chapters will define:

- Technology Stack
- Flutter Project Structure
- Core Platform Engines
- Database Design
- Content Package Format
- Navigation Architecture
- Design System
- Performance Strategy
- Development Standards

These chapters collectively provide the implementation blueprint for the Makeni Prayer Book: Catholic Companion platform.

Technical Requirements Document (TRD)

Chapter 2 — Technology Stack & Development Environment

Version: 1.0

1. Purpose

This chapter defines the approved technologies, frameworks, libraries, development tools, and engineering standards for the Makeni Prayer Book: Catholic Companion platform.

The objective is to maintain a modern, maintainable, high-performance, and offline-first codebase while minimizing unnecessary dependencies.

Only approved technologies listed in this document shall be used unless formally reviewed and accepted during development.

2. Core Technology Stack

Layer	Technology
Mobile Framework	Flutter (Latest Stable LTS)
Programming Language	Dart
IDE	Android Studio / VS Code
Version Control	Git + GitHub
Architecture	Clean Architecture + Feature-First
Design System	Material 3 (Customized)

3. State Management

Approved Technology

Riverpod (with Code Generation)

Responsibilities

- State Management
- Dependency Injection
- Business Logic Coordination
- Reactive Updates

- Application-wide State

Riverpod shall be the single state management solution across the application.

No additional state management frameworks shall be introduced.

4. Navigation

Approved Technology

Go Router

Responsibilities

- Declarative Routing
 - Deep Linking (Future)
 - Nested Navigation
 - Route Guards (Future)
 - Navigation History
 - Intelligent Navigation Engine Integration
-

5. Local Database

Approved Technology

Drift + SQLite

Responsibilities

- Structured Content Storage
- User Data
- Search Indexes
- Reading History
- Favorites
- Bookmarks
- Journal

- Notes
- Settings

SQLite is the storage engine.

Drift provides type-safe database access, migrations, and query generation.

6. Local Storage

SharedPreferences

Used for:

- UI Preferences
 - Theme
 - Font Size
 - Last Selected Tab
 - Simple Settings
-

Flutter Secure Storage

Used for:

- Encryption Keys
- Consent Information
- Future Authentication Tokens
- Sensitive Configuration

Sensitive information shall never be stored in SharedPreferences.

7. Backend Services

Firestore

Firebase shall be used only for optional cloud services.

Approved services include:

- Firebase Analytics (Opt-in)
- Firebase Crashlytics (Opt-in)
- Firebase Remote Config (Future)
- Firebase App Distribution
- Firebase Cloud Messaging (Future)

Firebase Authentication shall not be implemented in Version 1.

Firestore and Realtime Database are not required for Version 1.

8. Notifications

flutter_local_notifications

Responsibilities:

- Prayer Reminders
- Routine Notifications
- Liturgical Events
- Feast Day Notifications

All notifications shall function offline.

9. Networking

Dio

Responsibilities:

- Optional Content Updates
- Feedback Submission
- Future API Integration
- Future Synchronization

Version 1 shall require minimal network communication.

10. Serialization

Approved technologies:

- json_serializable
- freezed

Responsibilities:

- Immutable Models
 - JSON Parsing
 - DTO Generation
 - Equality
 - Copy Methods
-

11. Code Generation

The project shall use:

- build_runner
- freezed
- json_serializable
- riverpod_generator
- drift_dev

Generated code shall never be modified manually.

12. Logging

Logger

Responsibilities:

- Debug Logging
- Error Logging
- Development Diagnostics

Production builds shall disable verbose logging.

13. Testing

Testing shall include:

Unit Testing

- flutter_test
- mocktail

Widget Testing

- flutter_test

Integration Testing

- integration_test

Business logic shall be tested independently from the UI wherever practical.

14. Linting & Code Quality

Approved packages:

- flutter_lints
- custom_analysis (optional)

Standards:

- Zero analyzer warnings before merge.
- Consistent formatting using dart format.
- Strong typing throughout the codebase.
- Avoid unnecessary dynamic types.

15. Assets

Supported asset types include:

- PNG
- SVG
- WebP
- JSON
- Markdown
- Audio (Future)

Fonts shall be bundled locally to ensure offline availability.

16. Development Environment

Minimum requirements:

- Flutter (Latest Stable)
- Dart SDK (Bundled)
- Android Studio or VS Code
- Android SDK
- Xcode (macOS for iOS builds)
- Git

Developers shall use consistent SDK versions defined in the project configuration.

17. CI/CD (Future)

The architecture shall support automated pipelines for:

- Static Analysis
- Unit Testing
- Widget Testing
- Integration Testing
- Android Build

- iOS Build
- Release Distribution

GitHub Actions is the preferred CI/CD platform.

18. Dependency Management

Guidelines:

- Prefer official Flutter packages.
 - Minimize third-party dependencies.
 - Review package maintenance before adoption.
 - Avoid duplicate functionality.
 - Remove unused packages promptly.
 - Keep dependencies updated on a regular schedule.
-

19. Engineering Principles

All development shall follow these principles:

- Offline First
 - Clean Architecture
 - Feature-First Organization
 - Composition over Inheritance
 - SOLID Principles
 - Immutable Data Models
 - Reusable Components
 - Privacy by Design
 - Accessibility by Default
 - Performance-Oriented Development
-

20. Approved Core Packages

Purpose	Package
State Management	flutter_riverpod, riverpod_annotation
Routing	go_router
Database	drift, sqlite3_flutter_libs
Database Code Generation	drift_dev
Local Storage	shared_preferences
Secure Storage	flutter_secure_storage
Notifications	flutter_local_notifications
Networking	dio
Models	freezed, json_serializable
Logging	logger
SVG Rendering	flutter_svg
Testing	flutter_test, integration_test, mocktail
Utilities	intl, collection, path_provider

Chapter Summary

The technology stack has been intentionally kept small, modern, and stable.

Every selected package serves a clear purpose, minimizing maintenance overhead while providing a solid foundation for an offline-first Catholic digital platform.

Additional dependencies shall only be introduced when they provide significant value and do not duplicate existing functionality.

Technical Requirements Document (TRD)

Chapter 3 — Project Structure & Flutter Architecture

Version: 1.0

1. Purpose

This chapter defines the project structure, architectural layers, module organization, coding standards, and dependency rules for the Makeni Prayer Book: Catholic Companion platform.

The objective is to create a scalable, maintainable, and testable Flutter codebase that supports future expansion without architectural changes.

2. Architectural Style

The application shall adopt a **Feature-First Clean Architecture**.

The project is organized into:

- **Core** – Shared infrastructure and platform services.
- **Features** – Independent functional modules.
- **Shared** – Reusable UI components and utilities.
- **Assets** – Static resources and content packages.

Each feature shall encapsulate its own presentation, domain, and data layers.

3. Project Structure

lib/

```
|
|
| └─ app/
|   |
|   | └─ app.dart
|   |
|   | └─ bootstrap.dart
|   |
|   └─ router/
|
|
| └─ core/
|   |
|   | └─ config/
|   |
|   | └─ database/
|   |
|   | └─ engines/
|   |
|   | └─ services/
|   |
|   | └─ models/
|   |
|   | └─ errors/
|   |
|   | └─ constants/
|   |
|   | └─ utils/
|   |
|   | └─ theme/
|   |
|   | └─ accessibility/
|   |
|   └─ analytics/
|
|
| └─ features/
|   |
|   | └─ home/
|   |
|   | └─ library/
|   |
|   | └─ reader/
|   |
|   └─ search/
```

```
| |— companion/
| |— workspace/
| |— reminders/
| |— settings/
| |— onboarding/ (optional)
|
|— shared/
| |— widgets/
| |— layouts/
| |— dialogs/
| |— animations/
| |— extensions/
| |— providers/
|
|— assets/
| |— content/
| |— icons/
| |— images/
| |— fonts/
| |— lottie/
|
|— main.dart
```

This structure shall remain consistent throughout the project.

4. Feature Module Structure

Every feature shall follow the same internal organization.

feature_name/

data/

|

|— datasources/

|— models/

|— repositories/

domain/

|

|— entities/

|— repositories/

|— usecases/

presentation/

|

|— pages/

|— widgets/

|— controllers/

|— providers/

This structure promotes consistency and simplifies onboarding for new developers.

5. Core Module

The `core/` directory contains functionality shared across the entire application.

It shall include:

- Configuration Engine
- Content Engine
- Search Engine
- Rule Engine
- Theme Engine
- Reminder Engine
- Recommendation Engine
- Accessibility Engine
- Analytics Engine
- Secure Storage
- Database
- Utility Classes
- Error Handling
- Constants

Business-specific logic shall not reside in the Core module.

6. Shared Module

The `shared/` directory contains reusable UI and helper components.

Examples include:

- Buttons
- Cards
- Dialogs
- Bottom Sheets
- Navigation Components

- Loading Indicators
- Empty States
- Error States
- Reusable Animations
- Extensions

Widgets in **shared/** shall remain generic and independent of any specific feature.

7. Engine Architecture

Platform engines are shared services responsible for cross-feature functionality.

Initial engines include:

- Content Engine
- Reading Engine
- Search Engine
- Intelligent Navigation Engine (INE)
- Recommendation Engine
- Rule Engine
- Reminder Engine
- Configuration Engine
- Theme Engine
- Typography Engine
- Accessibility Engine
- Consent Engine
- Analytics Engine

Each engine shall expose well-defined public interfaces and avoid direct UI dependencies.

8. Dependency Rules

The following dependency rules shall be enforced:

- Features may depend on Core.
- Features may depend on Shared.
- Shared may depend on Core.
- Core shall not depend on Features.

- Features shall not depend directly on other Features.
- Communication between features shall occur through Core services or shared interfaces.

This prevents circular dependencies and preserves modularity.

9. State Management

Riverpod shall be the single state management solution.

Each feature shall define only the providers it owns.

Global application state shall reside within Core services when shared across multiple features.

10. Routing

Navigation shall be managed through Go Router.

Each feature shall register its own routes.

The application router shall aggregate all feature routes into a single navigation configuration.

Future deep-link support shall integrate through the same routing mechanism.

11. Data Flow

All features shall follow the same data flow.

User Interaction



Presentation Layer



Use Case



Repository Interface



Repository Implementation



Local Database / Service



Result



Presentation Update

This separation ensures maintainability and testability.

12. Naming Conventions

The following conventions shall be used:

- Classes: PascalCase
- Variables: camelCase
- Files: snake_case
- Directories: snake_case
- Providers: `xxxProvider`
- Controllers: `xxxController`
- Repositories: `xxxRepository`
- Use Cases: Verb-based (e.g., `SearchPrayers`, `LoadLibrary`)

Consistency is mandatory across the codebase.

13. Code Organization Principles

Developers shall:

- Keep files focused on a single responsibility.
 - Avoid business logic in widgets.
 - Prefer composition over inheritance.
 - Keep widgets small and reusable.
 - Favor immutable models.
 - Use dependency injection through Riverpod.
-

14. Scalability

New features shall be added by creating a new feature module without modifying the overall architecture.

Future modules may include:

- Bible
- Catechism
- Daily Readings
- Rosary
- Saints
- Parish Resources
- Audio
- Family Profiles

The architecture shall support these additions with minimal impact on existing code.

15. Chapter Summary

The Feature-First Clean Architecture provides a modular, scalable foundation for the Makeni Prayer Book: Catholic Companion.

By enforcing consistent project structure, dependency rules, and reusable platform engines, the application can evolve from a single prayer book into a comprehensive Catholic digital platform without requiring architectural redesign.

Technical Requirements Document (TRD)

Chapter 4 — Design System & UI Architecture

Version: 1.0

1. Purpose

This chapter defines the visual identity, design language, component standards, and user experience principles for Makeni Prayer Book: Catholic Companion.

The objective is to create an interface that is calm, elegant, intuitive, and accessible to users of all ages and technical abilities.

The design philosophy is **Sacred Simplicity**—reducing visual noise so users can focus on prayer rather than the interface.

2. Design Philosophy

The application shall follow these core principles:

- Simplicity over complexity.
- Clarity over decoration.
- Consistency over novelty.
- Accessibility by default.
- Calm, distraction-free interactions.
- Content-first design.
- Progressive disclosure (show only what is needed).
- Large, touch-friendly controls.
- Minimal cognitive load.

Every screen shall have a single, obvious primary action.

3. Visual Identity

The interface shall present a premium, modern aesthetic inspired by high-quality reading applications.

Characteristics include:

- Clean layouts with generous spacing.
- Rounded corners.
- Soft shadows used sparingly.
- Large readable typography.
- Limited color palette.
- Subtle animations.
- Card-based organization.
- Minimal iconography.
- Consistent spacing and alignment.

The interface shall never resemble a dense PDF viewer or a traditional document reader.

4. Color System

The default theme shall be warm, calm, and comfortable for extended reading.

Primary

Deep Royal Blue

Used for:

- Primary actions
 - Active navigation
 - Links
 - Selected items
-

Secondary

Warm Gold

Used for:

- Highlights
 - Liturgical accents
 - Important badges
 - Seasonal indicators
-

Background

Soft Off-White

Primary reading surface designed to reduce eye strain.

Surface

Pure White

Used for cards, dialogs, sheets, and elevated content.

Text

Near Black

Optimized for readability while avoiding the harshness of pure black.

Success

Muted Green

Warning

Amber

Error

Soft Red

Dark Theme

The dark theme shall use charcoal tones instead of pure black to improve readability, with an optional AMOLED mode for users who prefer true black backgrounds.

5. Typography

Typography is the foundation of the reading experience.

The design shall prioritize:

- Large default text.
- High readability.
- Comfortable line spacing.
- Clear hierarchy.
- Consistent heading styles.

Suggested scale:

- Display
- Heading 1
- Heading 2
- Heading 3
- Body Large
- Body
- Caption

Users shall be able to adjust font size without breaking layouts.

6. Spacing System

A consistent spacing scale shall be used throughout the application.

Preferred spacing values:

- 4 px
- 8 px

- 12 px
- 16 px
- 24 px
- 32 px
- 48 px

Spacing shall be consistent across all screens and components.

7. Layout Principles

Every screen shall include:

- A clear title.
- One primary focus.
- Minimal scrolling where practical.
- Predictable navigation.
- Consistent margins.
- Comfortable reading width.

Avoid placing multiple unrelated actions on the same screen.

8. Component Design

Reusable components shall include:

- Buttons
- Cards
- List Items
- Search Bar
- Bottom Sheets
- Dialogs
- Chips
- Badges
- Tabs
- Navigation Bar

Components shall maintain consistent shapes, spacing, and behavior across the application.

9. Navigation Design

Navigation shall prioritize discoverability and simplicity.

Primary destinations:

- Home
- Library
- Search
- Workspace
- Settings

Frequently used actions shall require as few taps as practical.

Navigation labels shall always accompany icons.

10. Reading Experience

The Reader shall provide:

- Distraction-free mode.
- Adjustable typography.
- Light, Dark, Sepia, and AMOLED themes.
- Smooth scrolling.
- Quick Jump support.
- Progress indicator.
- Continue Reading.

Reader controls shall fade when not in use.

11. Motion & Animation

Animations shall be subtle and purposeful.

Examples include:

- Page transitions.
- Card elevation.
- Search result appearance.
- Theme transitions.
- Button feedback.

Animations shall never delay user interactions.

A "Reduce Motion" option shall disable non-essential animations.

12. Accessibility

The interface shall support:

- Large touch targets.
- Dynamic text scaling.
- Screen readers.
- High contrast mode.
- Reduced motion.
- Haptic feedback (where supported).

Accessibility features shall be integrated into the design from the outset.

13. Empty States & Error States

Empty states shall provide clear guidance and encourage the next action.

Error messages shall be concise, understandable, and actionable.

The application shall avoid technical jargon in user-facing messages.

14. Responsive Design

The interface shall adapt gracefully to:

- Small phones.
- Large phones.
- Tablets.
- Foldable devices (future).

Layouts shall preserve readability regardless of screen size.

15. Design Consistency

All new screens and features shall conform to this design system.

Reusable components shall be used wherever possible to maintain a consistent visual identity and reduce development effort.

Chapter Summary

The Design System establishes a calm, minimalist, and accessible visual language centered on reading and prayer.

By emphasizing simplicity, consistency, and readability, the interface supports users of all ages and technical backgrounds while providing a premium experience that can grow with the platform.

Technical Requirements Document (TRD)

Chapter 4 — Design System & UI Architecture

Version: 1.0

1. Purpose

This chapter defines the visual identity, design language, component standards, and user experience principles for Makeni Prayer Book: Catholic Companion.

The objective is to create an interface that is calm, elegant, intuitive, and accessible to users of all ages and technical abilities.

The design philosophy is **Sacred Simplicity**—reducing visual noise so users can focus on prayer rather than the interface.

2. Design Philosophy

The application shall follow these core principles:

- Simplicity over complexity.
- Clarity over decoration.
- Consistency over novelty.
- Accessibility by default.
- Calm, distraction-free interactions.
- Content-first design.
- Progressive disclosure (show only what is needed).
- Large, touch-friendly controls.
- Minimal cognitive load.

Every screen shall have a single, obvious primary action.

3. Visual Identity

The interface shall present a premium, modern aesthetic inspired by high-quality reading applications.

Characteristics include:

- Clean layouts with generous spacing.
- Rounded corners.
- Soft shadows used sparingly.
- Large readable typography.

- Limited color palette.
- Subtle animations.
- Card-based organization.
- Minimal iconography.
- Consistent spacing and alignment.

The interface shall never resemble a dense PDF viewer or a traditional document reader.

4. Color System

The default theme shall be warm, calm, and comfortable for extended reading.

Primary

Deep Royal Blue

Used for:

- Primary actions
 - Active navigation
 - Links
 - Selected items
-

Secondary

Warm Gold

Used for:

- Highlights
 - Liturgical accents
 - Important badges
 - Seasonal indicators
-

Background

Soft Off-White

Primary reading surface designed to reduce eye strain.

Surface

Pure White

Used for cards, dialogs, sheets, and elevated content.

Text

Near Black

Optimized for readability while avoiding the harshness of pure black.

Success

Muted Green

Warning

Amber

Error

Soft Red

Dark Theme

The dark theme shall use charcoal tones instead of pure black to improve readability, with an optional AMOLED mode for users who prefer true black backgrounds.

5. Typography

Typography is the foundation of the reading experience.

The design shall prioritize:

- Large default text.
- High readability.
- Comfortable line spacing.
- Clear hierarchy.
- Consistent heading styles.

Suggested scale:

- Display
- Heading 1
- Heading 2
- Heading 3
- Body Large
- Body
- Caption

Users shall be able to adjust font size without breaking layouts.

6. Spacing System

A consistent spacing scale shall be used throughout the application.

Preferred spacing values:

- 4 px
- 8 px
- 12 px
- 16 px
- 24 px
- 32 px
- 48 px

Spacing shall be consistent across all screens and components.

7. Layout Principles

Every screen shall include:

- A clear title.
- One primary focus.
- Minimal scrolling where practical.
- Predictable navigation.
- Consistent margins.
- Comfortable reading width.

Avoid placing multiple unrelated actions on the same screen.

8. Component Design

Reusable components shall include:

- Buttons
- Cards
- List Items
- Search Bar
- Bottom Sheets
- Dialogs
- Chips
- Badges
- Tabs
- Navigation Bar

Components shall maintain consistent shapes, spacing, and behavior across the application.

9. Navigation Design

Navigation shall prioritize discoverability and simplicity.

Primary destinations:

- Home
- Library

- Search
- Workspace
- Settings

Frequently used actions shall require as few taps as practical.

Navigation labels shall always accompany icons.

10. Reading Experience

The Reader shall provide:

- Distraction-free mode.
- Adjustable typography.
- Light, Dark, Sepia, and AMOLED themes.
- Smooth scrolling.
- Quick Jump support.
- Progress indicator.
- Continue Reading.

Reader controls shall fade when not in use.

11. Motion & Animation

Animations shall be subtle and purposeful.

Examples include:

- Page transitions.
- Card elevation.
- Search result appearance.
- Theme transitions.
- Button feedback.

Animations shall never delay user interactions.

A "Reduce Motion" option shall disable non-essential animations.

12. Accessibility

The interface shall support:

- Large touch targets.
- Dynamic text scaling.
- Screen readers.
- High contrast mode.
- Reduced motion.
- Haptic feedback (where supported).

Accessibility features shall be integrated into the design from the outset.

13. Empty States & Error States

Empty states shall provide clear guidance and encourage the next action.

Error messages shall be concise, understandable, and actionable.

The application shall avoid technical jargon in user-facing messages.

14. Responsive Design

The interface shall adapt gracefully to:

- Small phones.
- Large phones.
- Tablets.
- Foldable devices (future).

Layouts shall preserve readability regardless of screen size.

15. Design Consistency

All new screens and features shall conform to this design system.

Reusable components shall be used wherever possible to maintain a consistent visual identity and reduce development effort.

Chapter Summary

The Design System establishes a calm, minimalist, and accessible visual language centered on reading and prayer.

By emphasizing simplicity, consistency, and readability, the interface supports users of all ages and technical backgrounds while providing a premium experience that can grow with the platform.

Technical Requirements Document (TRD)

Chapter 5 — Core Platform Architecture

Version: 1.0

1. Purpose

This chapter defines the shared platform architecture used throughout the Makeni Prayer Book: Catholic Companion application.

Instead of duplicating business logic across features, the application is built around reusable platform engines. Each engine has a single responsibility and can be reused by multiple features.

This modular approach improves maintainability, scalability, testing, and long-term evolution of the platform.

2. Architectural Overview

The platform is divided into three layers:

Feature Modules

User-facing functionality such as:

- Home
- Library
- Reader
- Search
- Prayer Companion
- Workspace
- Settings

These modules focus only on presentation and user interactions.

Domain Engines

Shared business logic that powers multiple features.

Examples include:

- Content Engine
- Search Engine
- Reading Engine
- Recommendation Engine
- Rule Engine
- Intelligent Navigation Engine

Domain Engines are independent of the UI.

Core Services

Platform infrastructure shared by the entire application.

Examples include:

- Database Service
- Configuration Service
- Secure Storage Service
- Notification Service
- Analytics Service
- Logging Service

Core Services provide technical capabilities but contain minimal business logic.

3. Domain Engines

Content Engine

Responsibility

- Load structured prayer content.
- Manage hymn content.
- Manage metadata.
- Serve content to features.
- Validate installed content packages.

Used by

- Library
 - Reader
 - Search
 - Prayer Companion
 - Recommendations
-

Reading Engine

Responsibility

- Continue reading.
- Reading progress.

- Reader preferences.
 - Theme integration.
 - Typography handling.
-

Search Engine

Responsibility

- Full-text search.
 - Prayer lookup.
 - Hymn number lookup.
 - Page lookup.
 - Keyword search.
 - Instant search suggestions.
 - Search ranking.
-

Intelligent Navigation Engine (INE)

Responsibility

- Quick Jump.
 - Navigation history.
 - Related content.
 - Continue reading.
 - Smart navigation between prayers and hymns.
-

Recommendation Engine

Responsibility

- Daily prayer suggestions.
- Seasonal recommendations.
- Recently used prayers.
- Frequently accessed hymns.
- Personalized suggestions.

Recommendations are generated locally to preserve privacy.

Rule Engine

Responsibility

- Liturgical calendar rules.
 - Feast day logic.
 - Recommendation rules.
 - Reminder rules.
 - Future content rules.
-

4. Core Services

The following infrastructure services support the platform:

- Configuration Service
- Notification Service
- Database Service
- Logging Service
- Secure Storage Service
- Analytics Service
- Content Package Service

These services expose reusable interfaces to the Domain Engines.

5. Engine Communication

Feature Modules communicate with Domain Engines.

Domain Engines communicate with Core Services.

Feature Modules shall never communicate directly with Core Services unless required for UI-specific functionality.

This separation minimizes coupling and improves testability.

6. Extensibility

New features shall integrate by reusing existing engines wherever possible.

If new business logic is shared across multiple modules, it shall be implemented as a new Domain Engine rather than duplicated.

Chapter Summary

The Core Platform Architecture establishes a modular foundation where reusable Domain Engines encapsulate business logic and Core Services provide infrastructure.

This design keeps features lightweight, promotes consistency across the application, and supports future expansion without requiring architectural redesign.

Technical Requirements Document (TRD)

Chapter 6 — UI Component Library & Design Standards

Version: 1.0

1. Purpose

This chapter defines the reusable UI components, interaction patterns, spacing rules, and interface standards used throughout the application.

The objective is to create a consistent, intuitive, and maintainable user interface by building every screen from a shared component library rather than designing each page independently.

2. Design Philosophy

Every component shall be:

- Simple.
- Consistent.
- Accessible.
- Reusable.
- Responsive.
- Touch-friendly.

Components shall prioritize readability over decoration.

3. Component Categories

The application shall use a standardized component library grouped into the following categories:

Navigation

- App Bar
 - Bottom Navigation Bar
 - Back Button
 - Search Bar
 - Breadcrumb (future)
-

Cards

- Prayer Card
- Hymn Card
- Recommendation Card
- Continue Reading Card
- Daily Prayer Card
- Liturgical Event Card

Cards shall use consistent padding, elevation, and corner radius.

Lists

- Prayer List Item
- Hymn List Item
- Search Result Item
- Bookmark Item
- History Item
- Setting Item

List items shall support icons, subtitles, badges, and trailing actions where appropriate.

Buttons

The application shall use only four button styles:

- Primary Button
- Secondary Button
- Text Button
- Icon Button

Button styles shall remain consistent across all screens.

Inputs

Input components include:

- Search Field
- Text Field
- Multiline Field
- Number Input (future)

Inputs shall provide clear labels and validation feedback.

Feedback Components

Reusable feedback components include:

- Loading Indicator
- Empty State
- Error State

- Success Message
- Snackbar
- Confirmation Dialog

Feedback shall always explain the next available action.

Sheets & Dialogs

Reusable overlays include:

- Bottom Sheet
- Modal Dialog
- Confirmation Dialog
- Action Sheet

These components shall follow consistent spacing and animation rules.

4. Icons

Icons shall:

- Be simple and universally recognizable.
- Always accompany text for primary navigation.
- Maintain consistent sizing throughout the application.

Icons shall never be the only means of conveying critical information.

5. Layout Standards

Each screen shall follow a predictable structure:

1. App Bar
2. Primary Content
3. Optional Floating Action
4. Bottom Navigation (where applicable)

Layouts shall maintain consistent margins and spacing.

6. Touch Targets

Interactive elements shall provide touch targets of at least **48 × 48 dp** to ensure comfortable use for all users.

7. Motion Standards

Animations shall:

- Be subtle.
- Reinforce user actions.
- Complete within approximately **150–250 ms**.
- Never delay access to content.

Users may disable non-essential animations through accessibility settings.

8. Empty States

Empty states shall:

- Explain why no content is shown.
- Provide a clear next action.
- Use friendly, encouraging language.

Examples include empty bookmarks, empty search results, and empty journal entries.

9. Loading States

The application shall prefer:

- Skeleton placeholders for lists.

- Progress indicators for longer operations.
- Instant transitions where data is already available locally.

Offline-first design should minimize visible loading wherever possible.

10. Accessibility Standards

All components shall:

- Support dynamic text scaling.
 - Meet recommended color contrast ratios.
 - Be compatible with screen readers.
 - Provide descriptive semantics.
 - Maintain keyboard accessibility where applicable.
-

11. Reusability Rules

No feature shall create a custom component if an equivalent reusable component already exists in the shared library.

Enhancements to shared components shall be preferred over duplication.

12. Chapter Summary

The UI Component Library ensures that every feature is built from a common set of reusable, accessible, and consistent building blocks.

This approach reduces development time, simplifies maintenance, and delivers a cohesive experience across the entire application.